

Exercise 1: Implementing the Singleton Pattern

Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Steps:

1. Create a New Java Project:

- Create a new Java project named **SingletonPatternExample**.

2. Define a Singleton Class:

- Create a class named `Logger` that has a private static instance of itself.
- Ensure the constructor of `Logger` is private.
- Provide a public static method to get the instance of the `Logger` class.

3. Implement the Singleton Pattern:

- Write code to ensure that the `Logger` class follows the Singleton design pattern.

4. Test the Singleton Implementation:

- Create a test class to verify that only one instance of `Logger` is created and used across the application.

```

class Logger {
    private static Logger instance;
    private Logger() {
        System.out.println("Logger Instance Created");
    }
    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }
    public void log(String message) {
        System.out.println("LOG: " + message);
    }
}

public class Main {
    public static void main(String[] args) {
        Logger logger1 = Logger.getInstance();
        logger1.log("First log message");
        Logger logger2 = Logger.getInstance();
        logger2.log("Second log message");
        if (logger1 == logger2) {
            System.out.println("Both logger objects are the same instance.");
        } else {
            System.out.println("Different instances created.");
        }
    }
}

```

```

Logger Instance Created
LOG: First log message
LOG: Second log message
Both logger objects are the same instance.

...Program finished with exit code 0
Press ENTER to exit console.

```

Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Steps:

1. Create a New Java Project:

- Create a new Java project named **FactoryMethodPatternExample**.

2. Define Document Classes:

- Create interfaces or abstract classes for different document types such as **WordDocument**, **PdfDocument**, and **ExcelDocument**.

3. Create Concrete Document Classes:

- Implement concrete classes for each document type that implements or extends the above interfaces or abstract classes.

4. Implement the Factory Method:

- Create an abstract class **DocumentFactory** with a method **createDocument()**.
- Create concrete factory classes for each document type that extends **DocumentFactory** and implements the **createDocument()** method.

5. Test the Factory Method Implementation:

- Create a test class to demonstrate the creation of different document types using the factory method.

```

interface Document {
    void open();
}
class WordDocument implements Document {
    public void open() {
        System.out.println("Opening Word Document.");
    }
}
class PdfDocument implements Document {
    public void open() {
        System.out.println("Opening PDF Document.");
    }
}
class ExcelDocument implements Document {
    public void open() {
        System.out.println("Opening Excel Document.");
    }
}
abstract class DocumentFactory {
    public abstract Document createDocument();
}
class WordDocumentFactory extends DocumentFactory {
    public Document createDocument() {
        return new WordDocument();
    }
}

```

```

class PdfDocumentFactory extends DocumentFactory {
    public Document createDocument() {
        return new PdfDocument();
    }
}
class ExcelDocumentFactory extends DocumentFactory {
    public Document createDocument() {
        return new ExcelDocument();
    }
}
public class Main {
    public static void main(String[] args) {
        DocumentFactory wordFactory = new WordDocumentFactory();
        Document word = wordFactory.createDocument();
        word.open();
        DocumentFactory pdfFactory = new PdfDocumentFactory();
        Document pdf = pdfFactory.createDocument();
        pdf.open();
        DocumentFactory excelFactory = new ExcelDocumentFactory();
        Document excel = excelFactory.createDocument();
        excel.open();
    }
}

```

```
Logger Instance Created
LOG: First log message
LOG: Second log message
Both logger objects are the same instance.

...Program finished with exit code 0
Press ENTER to exit console.█
```

Exercise 3: Implementing the Builder Pattern

Scenario:

You are developing a system to create complex objects such as a Computer with multiple optional parts. Use the Builder Pattern to manage the construction process.

Steps:

1. Create a New Java Project:

- Create a new Java project named **BuilderPatternExample**.

2. Define a Product Class:

- Create a class **Computer** with attributes like **CPU**, **RAM**, **Storage**, etc.

3. Implement the Builder Class:

- Create a static nested Builder class inside Computer with methods to set each attribute.
- Provide a **build()** method in the Builder class that returns an instance of Computer.

4. Implement the Builder Pattern:

- Ensure that the **Computer** class has a private constructor that takes the **Builder** as a parameter.

5. Test the Builder Implementation:

- Create a test class to demonstrate the creation of different configurations of Computer using the Builder pattern.

```

class Computer {
    private String CPU;
    private int RAM;
    private int Storage;
    private String GraphicsCard;
    private String OperatingSystem;
    private Computer(Builder builder) {
        this.CPU = builder.CPU;
        this.RAM = builder.RAM;
        this.Storage = builder.Storage;
        this.GraphicsCard = builder.GraphicsCard;
        this.OperatingSystem = builder.OperatingSystem;
    }
    public void showConfiguration() {
        System.out.println("Computer Configuration:");
        System.out.println("CPU: " + CPU);
        System.out.println("RAM: " + RAM + " GB");
        System.out.println("Storage: " + Storage + " GB");
        System.out.println("Graphics Card: " + GraphicsCard);
        System.out.println("Operating System: " + OperatingSystem);
        System.out.println();
    }
    public static class Builder {
        private String CPU;
        private int RAM;
        private int Storage;

```

```

        private String GraphicsCard;
        private String OperatingSystem;
        public Builder setCPU(String CPU) {
            this.CPU = CPU;
            return this;
        }
        public Builder setRAM(int RAM) {
            this.RAM = RAM;
            return this;
        }
        public Builder setStorage(int Storage) {
            this.Storage = Storage;
            return this;
        }
        public Builder setGraphicsCard(String GraphicsCard) {
            this.GraphicsCard = GraphicsCard;
            return this;
        }
        public Builder setOperatingSystem(String OperatingSystem) {
            this.OperatingSystem = OperatingSystem;
            return this;
        }
        public Computer build() {
            return new Computer(this);
        }
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Computer gamingPC = new Computer.Builder()
            .setCPU("Intel Core i9")
            .setRAM(32)
            .setStorage(1000)
            .setGraphicsCard("NVIDIA RTX 4080")
            .setOperatingSystem("Windows 11")
            .build();
        Computer officePC = new Computer.Builder()
            .setCPU("Intel Core i5")
            .setRAM(16)
            .setStorage(512)
            .setGraphicsCard("Integrated Graphics")
            .setOperatingSystem("Windows 10")
            .build();
        gamingPC.showConfiguration();
        officePC.showConfiguration();
    }
}

```

```

Graphics Card: NVIDIA RTX 4080
Operating System: Windows 11

```

```

Computer Configuration:
CPU: Intel Core i5
RAM: 16 GB
Storage: 512 GB
Graphics Card: Integrated Graphics
Operating System: Windows 10

```

Exercise 4: Implementing the Adapter Pattern

Scenario:

You are developing a payment processing system that needs to integrate with multiple third-party payment gateways with different interfaces. Use the Adapter Pattern to achieve this.

Steps:

1. Create a New Java Project:

- Create a new Java project named **AdapterPatternExample**.

2. Define Target Interface:

- Create an interface **PaymentProcessor** with methods like **processPayment()**.

3. Implement Adaptee Classes:

- Create classes for different payment gateways with their own methods.

4. Implement the Adapter Class:

- Create an adapter class for each payment gateway that implements **PaymentProcessor** and translates the calls to the gateway-specific methods.

5. Test the Adapter Implementation:

- Create a test class to demonstrate the use of different payment gateways through the adapter.

```
interface PaymentProcessor {
    void processPayment();
}
class PayPal {
    public void makePayment() {
        System.out.println("Payment made using PayPal");
    }
}
class PayPalAdapter implements PaymentProcessor {
    private PayPal paypal = new PayPal();
    public void processPayment() {
        paypal.makePayment();
    }
}
public class Main {
    public static void main(String[] args) {
        PaymentProcessor payment = new PayPalAdapter();
        payment.processPayment();
    }
}
```

```
Payment made using PayPal
```

Exercise 5: Implementing the Decorator Pattern

Scenario:

You are developing a notification system where notifications can be sent via multiple channels (e.g., Email, SMS). Use the Decorator Pattern to add functionalities dynamically.

Steps:

1. Create a New Java Project:

- Create a new Java project named `DecoratorPatternExample`.

2. Define Component Interface:

- Create an interface `Notifier` with a method `send()`.

3. Implement Concrete Component:

- Create a class `EmailNotifier` that implements `Notifier`.

4. Implement Decorator Classes:

- Create abstract decorator class `NotifierDecorator` that implements `Notifier` and holds a reference to a `Notifier` object.
- Create concrete decorator classes like `SMSNotifierDecorator`, `SlackNotifierDecorator` that extend `NotifierDecorator`.

5. Test the Decorator Implementation:

- Create a test class to demonstrate sending notifications via multiple channels using decorators.

```

interface Notifier {
    void send();
}
class EmailNotifier implements Notifier {
    public void send() {
        System.out.println("Sending Email Notification");
    }
}
abstract class NotifierDecorator implements Notifier {
    protected Notifier notifier;
    public NotifierDecorator(Notifier notifier) {
        this.notifier = notifier;
    }
    public void send() {
        notifier.send();
    }
}
class SMSNotifierDecorator extends NotifierDecorator {
    public SMSNotifierDecorator(Notifier notifier) {
        super(notifier);
    }
    public void send() {
        super.send();
        System.out.println("Sending SMS Notification");
    }
}

```

```

class SlackNotifierDecorator extends NotifierDecorator {
    public SlackNotifierDecorator(Notifier notifier) {
        super(notifier);
    }
    public void send() {
        super.send();
        System.out.println("Sending Slack Notification");
    }
}

public class Main {
    public static void main(String[] args) {
        Notifier notifier = new SlackNotifierDecorator(
            new SMSNotifierDecorator(
                new EmailNotifier()));
        notifier.send();
    }
}

```

```
Sending Email Notification  
Sending SMS Notification  
Sending Slack Notification
```

Exercise 7: Implementing the Observer Pattern

Scenario:

You are developing a stock market monitoring application where multiple clients need to be notified whenever stock prices change. Use the Observer Pattern to achieve this.

Steps:

1. Create a New Java Project:

- Create a new Java project named `ObserverPatternExample`.

2. Define Subject Interface:

- Create an interface `Stock` with methods to register, deregister, and notify observers.

3. Implement Concrete Subject:

- Create a class `StockMarket` that implements `Stock` and maintains a list of observers.

4. Define Observer Interface:

- Create an interface `Observer` with a method `update()`.

5. Implement Concrete Observers:

- Create classes `MobileApp`, `WebApp` that implement `Observer`.

6. Test the Observer Implementation:

- Create a test class to demonstrate the registration and notification of observers.

```
import java.util.*;
interface Observer {
    void update(String price);
}
interface Stock {
    void register(Observer o);
    void deregister(Observer o);
    void notifyObservers();
}
class StockMarket implements Stock {
    private List<Observer> observers = new ArrayList<>();
    private String stockPrice;
    public void register(Observer o) {
        observers.add(o);
    }
    public void deregister(Observer o) {
        observers.remove(o);
    }
    public void setPrice(String price) {
        stockPrice = price;
        notifyObservers();
    }
    public void notifyObservers() {
        for (Observer o : observers) {
            o.update(stockPrice);
        }
    }
}
```

```

class MobileApp implements Observer {
    public void update(String price) {
        System.out.println("Mobile App: Stock Price = " + price);
    }
}
class WebApp implements Observer {
    public void update(String price) {
        System.out.println("Web App: Stock Price = " + price);
    }
}
public class Main {
    public static void main(String[] args) {
        StockMarket stock = new StockMarket();
        Observer mobile = new MobileApp();
        Observer web = new WebApp();
        stock.register(mobile);
        stock.register(web);
        stock.setPrice("Rs.500");
    }
}

```

```

Mobile App: Stock Price = Rs.500
Web App: Stock Price = Rs.500

```

Exercise 8: Implementing the Strategy Pattern

Scenario:

You are developing a payment system where different payment methods (e.g., Credit Card, PayPal) can be selected at runtime. Use the Strategy Pattern to achieve this.

Steps:

1. Create a New Java Project:

- Create a new Java project named StrategyPatternExample.

2. Define Strategy Interface:

- Create an interface PaymentStrategy with a method pay().

3. Implement Concrete Strategies:

- Create classes CreditCardPayment, PayPalPayment that implement PaymentStrategy.

4. Implement Context Class:

- Create a class `PaymentContext` that holds a reference to `PaymentStrategy` and a method to execute the strategy.

5. Test the Strategy Implementation:

- Create a test class to demonstrate selecting and using different payment strategies.

```
interface PaymentStrategy {
    void pay(int amount);
}

class CreditCardPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid Rs." + amount + " using Credit Card");
    }
}

class PayPalPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid Rs." + amount + " using PayPal");
    }
}

class PaymentContext {
    private PaymentStrategy strategy;
    public PaymentContext(PaymentStrategy strategy) {
        this.strategy = strategy;
    }
    public void makePayment(int amount) {
        strategy.pay(amount);
    }
}

public class Main {
    public static void main(String[] args) {
        PaymentContext payment;
        payment = new PaymentContext(new CreditCardPayment());
        payment.makePayment(1000);
        payment = new PaymentContext(new PayPalPayment());
        payment.makePayment(2000);
    }
}
```

```
Paid Rs.1000 using Credit Card
Paid Rs.2000 using PayPal
```

Exercise 9: Implementing the Command Pattern

Scenario: You are developing a home automation system where commands can be issued to turn devices on or off. Use the Command Pattern to achieve this.

Steps:

1. Create a New Java Project:

- Create a new Java project named **CommandPatternExample**.

2. Define Command Interface:

- Create an interface **Command** with a method **execute()**.

3. Implement Concrete Commands:

- Create classes **LightOnCommand**, **LightOffCommand** that implement **Command**.

4. Implement Invoker Class:

- Create a class **RemoteControl** that holds a reference to a **Command** and a method to execute the command.

5. Implement Receiver Class:

- Create a class **Light** with methods to turn on and off.

6. Test the Command Implementation:

- Create a test class to demonstrate issuing commands using the **RemoteControl**.

```
interface Command {
    void execute();
}

class Light {
    public void on() {
        System.out.println("Light is ON");
    }
    public void off() {
        System.out.println("Light is OFF");
    }
}

class LightOnCommand implements Command {
    private Light light;
    public LightOnCommand(Light light) {
        this.light = light;
    }
    public void execute() {
        light.on();
    }
}

class LightOffCommand implements Command {
    private Light light;
    public LightOffCommand(Light light) {
        this.light = light;
    }
    public void execute() {
        light.off();
    }
}
```



```

    }
}
class RemoteControl {
    private Command command;
    public void setCommand(Command command) {
        this.command = command;
    }
    public void pressButton() {
        command.execute();
    }
}
public class Main {
    public static void main(String[] args) {
        Light light = new Light();
        Command on = new LightOnCommand(light);
        Command off = new LightOffCommand(light);
        RemoteControl remote = new RemoteControl();
        remote.setCommand(on);
        remote.pressButton();
        remote.setCommand(off);
        remote.pressButton();
    }
}

```

```

Light is ON
Light is OFF

```

Exercise 10: Implementing the MVC Pattern

Scenario:

You are developing a simple web application for managing student records using the MVC pattern.

Steps:

1. Create a New Java Project:

- Create a new Java project named MVCPatternExample.

2. Define Model Class:

- Create a class Student with attributes like name, id, and grade.

3. Define View Class:

- Create a class **StudentView** with a method **displayStudentDetails()**.

4. Define Controller Class:

- Create a class **StudentController** that handles the communication between the model and the view.

5. Test the MVC Implementation:

- Create a main class to demonstrate creating a **Student**, updating its details using **StudentController**, and displaying them using **StudentView**.

```
class Student {  
    private String name;  
    private int id;  
    private String grade;  
    public Student(String name, int id, String grade) {  
        this.name = name;  
        this.id = id;  
        this.grade = grade;  
    }  
    public String getName() {  
        return name;  
    }  
    public int getId() {  
        return id;  
    }  
    public String getGrade() {  
        return grade;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public void setGrade(String grade) {  
        this.grade = grade;  
    }  
}
```

```
class StudentView {
    public void displayStudentDetails(Student student) {
        System.out.println("Student Details");
        System.out.println("Name : " + student.getName());
        System.out.println("ID : " + student.getId());
        System.out.println("Grade : " + student.getGrade());
    }
}

class StudentController {
    private Student model;
    private StudentView view;
    public StudentController(Student model, StudentView view) {
        this.model = model;
        this.view = view;
    }
    public void setStudentName(String name) {
        model.setName(name);
    }
    public void setStudentGrade(String grade) {
        model.setGrade(grade);
    }
    public void updateView() {
        view.displayStudentDetails(model);
    }
}

public class Main {
    public static void main(String[] args) {
        Student student = new Student("Shreyasi", 9030, "A");
    }
}
```

```

Student student = new Student("Shreyasi", 9030, "A");
StudentView view = new StudentView();
StudentController controller = new StudentController(student, view);
controller.updateView();
System.out.println();
controller.setStudentName("Priya");
controller.setStudentGrade("A+");
controller.updateView();
}

```

```

Student Details
Name : Shreyasi
ID : 9030
Grade : A

Student Details
Name : Priya
ID : 9030
Grade : A+

```

Exercise 11: Implementing Dependency Injection

Scenario:

You are developing a customer management application where the service class depends on a repository class. Use Dependency Injection to manage these dependencies.

Steps:

1. Create a New Java Project:

- Create a new Java project named **DependencyInjectionExample**.

2. Define Repository Interface:

- Create an interface **CustomerRepository** with methods like **findCustomerById()**.

3. Implement Concrete Repository:

- Create a class **CustomerRepositoryImpl** that implements **CustomerRepository**.

4. Define Service Class:

- Create a class **CustomerService** that depends on **CustomerRepository**.

5. Implement Dependency Injection:

- Use constructor injection to inject **CustomerRepository** into **CustomerService**.

6. Test the Dependency Injection Implementation:

- Create a main class to demonstrate creating a **CustomerService** with **CustomerRepositoryImpl** and using it to find a customer.

```
interface CustomerRepository {
    String findCustomerById(int id);
}
class CustomerRepositoryImpl implements CustomerRepository {
    public String findCustomerById(int id) {
        return "Customer ID: " + id + ", Name: Shreyasi";
    }
}
class CustomerService {
    private CustomerRepository repository;
    public CustomerService(CustomerRepository repository) {
        this.repository = repository;
    }
    public void getCustomer(int id) {
        System.out.println(repository.findCustomerById(id));
    }
}
public class Main {
    public static void main(String[] args) {
        CustomerRepository repository = new CustomerRepositoryImpl();
        CustomerService service = new CustomerService(repository);
        service.getCustomer(101);
    }
}
```

```
Customer ID: 101, Name: Shreyasi
```